

My Content

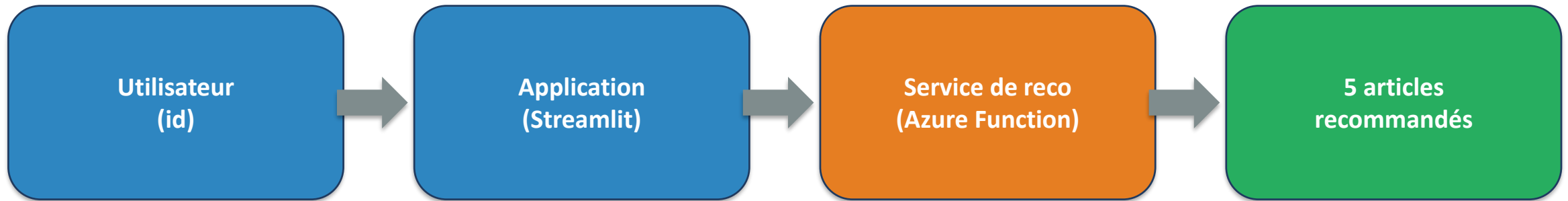
MVP — Système de recommandation d'articles

Présentation à Samia (CEO) — support technique & architecture

Contexte & objectif

- My Content : start-up qui veut encourager la lecture par la recommandation de contenus.
- Pas encore de données propriétaires → jeu de données public (interactions Globo.com).
- Fonctionnalité MVP la plus critique :
 - « En tant qu'utilisateur, je reçois une sélection de 5 articles. »
- Contrainte structurante : anticiper l'ajout de nouveaux utilisateurs et de nouveaux articles.
- Exigence : solution serverless, industrialisable, déployable de bout en bout.

Description fonctionnelle de l'application



- L'app liste des identifiants utilisateurs et déclenche l'appel du service.
- Le service serverless reçoit un `user_id` et renvoie le top-5 des articles.
- Objectif : démontrer la fonctionnalité à Samia et à de futurs utilisateurs.

Les données

Globo.com — interactions utilisateurs $\leftrightarrow$ articles

- Clics / sessions : user_id, session, article cliqué, horodatage.
- Articles : catégorie, date de création, éditeur, nombre de mots.
- Embeddings : vecteur de 250 dimensions par article (contenu).
- Nature : feedback implicite (clics), pas de notes explicites.
- Enjeux : forte creusité (users/articles peu actifs), volumétrie des embeddings.

Approches de recommandation analysées

Content-based

Similarité de contenu
(embeddings d'articles)

**Collaborative
filtering (ALS)**

Goûts d'utilisateurs
similaires

Hybride

Combinaison des deux
+ fallback popularité

- Les trois stratégies sont exposées derrière un même service ; l'hybride est la stratégie par défaut.

Modèle 1 — Filtrage basé sur le contenu

- Principe : profil utilisateur = moyenne des embeddings des articles lus, puis similarité cosinus vers tout le catalogue.

Avantages

- Fonctionne dès le 1^{er} clic (peu de cold start utilisateur)
- Intègre un nouvel article immédiatement (il a un embedding)
- Explicable : « proche de ce que vous avez lu »

Inconvénients

- N'exploite pas l'intelligence collective
- Risque de « bulle » (peu de diversité/sérendipité)
- Dépend de la qualité des embeddings

Modèle 2 — Filtrage collaboratif (ALS)

- Principe : factorisation de la matrice utilisateur $\times$ article (feedback implicite) par ALS $\rightarrow$ facteurs latents.

Avantages

- Capte des goûts latents non visibles dans le contenu
- Effet de découverte / sérendipité
- Performant sur les utilisateurs actifs

Inconvénients

- Cold start fort (nouvel utilisateur ET nouvel article)
- Ré-entraînement nécessaire (ALS non incrémental)
- Sensible à la creusité des données

Modèle 3 — Hybride + cold start

- Principe : combinaison des scores content + collaboratif (normalisés) ; repli sur les articles populaires si historique

Avantages

- Combine les forces des deux approches
- Dégradation propre : popularité en dernier recours
- Couvre nouveaux utilisateurs et nouveaux articles

Inconvénients

- Deux modèles à maintenir et à synchroniser
- Pondération à régler (paramètre alpha)
- Un peu plus de complexité opérationnelle

Comparaison des modèles

Critère	Content-based	Collaboratif (ALS)	Hybride
Nouvel utilisateur	Bon (dès 1 clic)	Faible	Bon
Nouvel article	Excellent	Faible	Bon
Sérendipité	Faible	Bonne	Bonne
Coût d'entraînement	Faible	Élevé (batch)	Moyen
Explicabilité	Élevée	Faible	Moyenne
Choix MVP	—	—	✓ retenu

- Évaluation par leave-last-out (HitRate@5) dans le notebook pour départager sur nos données.

Système de recommandation retenu

- Stratégie hybride par défaut, exposée via le service serverless.
- Chaîne de décision pour un user_id :
 - 1. Historique exploitable → score hybride (content + collaboratif)
 - 2. Historique faible → content-based seul
 - 3. Aucun historique → articles les plus populaires (cold start)
- Inférence légère : ne dépend que de numpy (artefacts pré-calculés hors-ligne).
- Renvoie le top-5 (paramétrable).

Optimisation production — réduction de dimension (ACP)

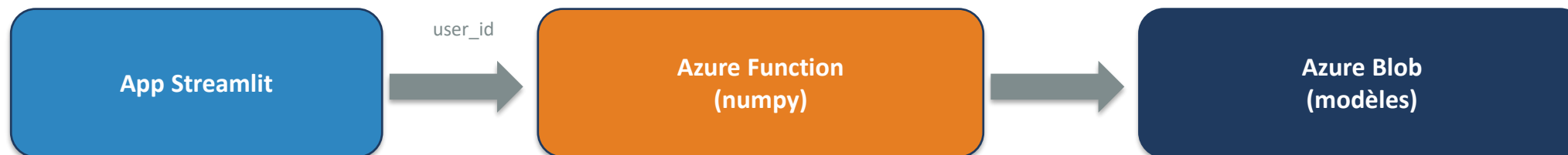


- Fichier d'embeddings volumineux vs limites du free tier Azure.
- Réduction par ACP hors-ligne → artefact plus léger, latence réduite.
- Compromis maîtrisé entre taille/temps et qualité des recommandations.

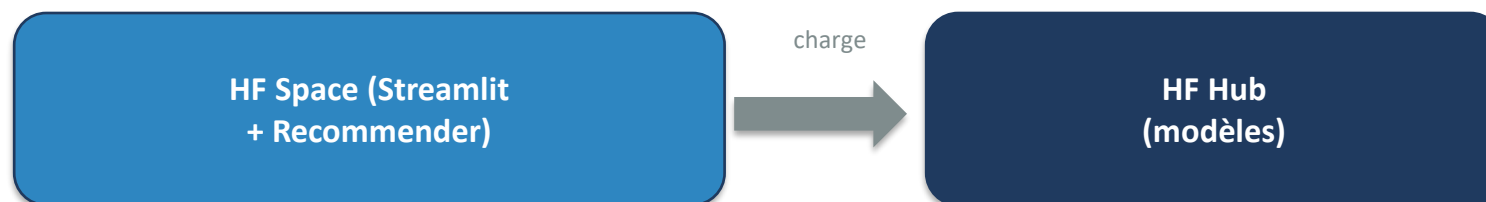
Architecture retenue — deux solutions indépendantes

Même cœur de reco et mêmes artefacts, deux déploiements autonomes

① Solution Azure — serverless



② Solution Hugging Face — démo publique



Pipeline hors-ligne COMMUN (prepare_model.py) : données brutes → ACP + ALS → artefacts publiés vers Blob ET HF Hub

Pourquoi l'Architecture 2 pour le MVP ?

Critère	Archi 1 — API dédiée	Archi 2 — Blob (retenue)
Pièces mobiles	API + Function	Function seule
Coût (free tier)	Plus élevé	Quasi nul
Mise en place	Plus longue	Rapide
Séparation des responsabilités	Meilleure	Suffisante (MVP)
Montée en charge	Meilleure (cible)	Limitée

- MVP : simplicité et coût priment → Archi 2. L'Archi 1 est visée pour l'architecture cible.

Démonstration — les applications

- Interface Streamlit : sélection d'un identifiant utilisateur → 5 articles.
- Deux vitrines, selon la solution :
 - Azure : app locale qui appelle le service serverless (Azure Function).
 - Hugging Face : Space public auto-suffisant (reco calculée sur place).
- Paramétrable : stratégie de reco, nombre d'articles.



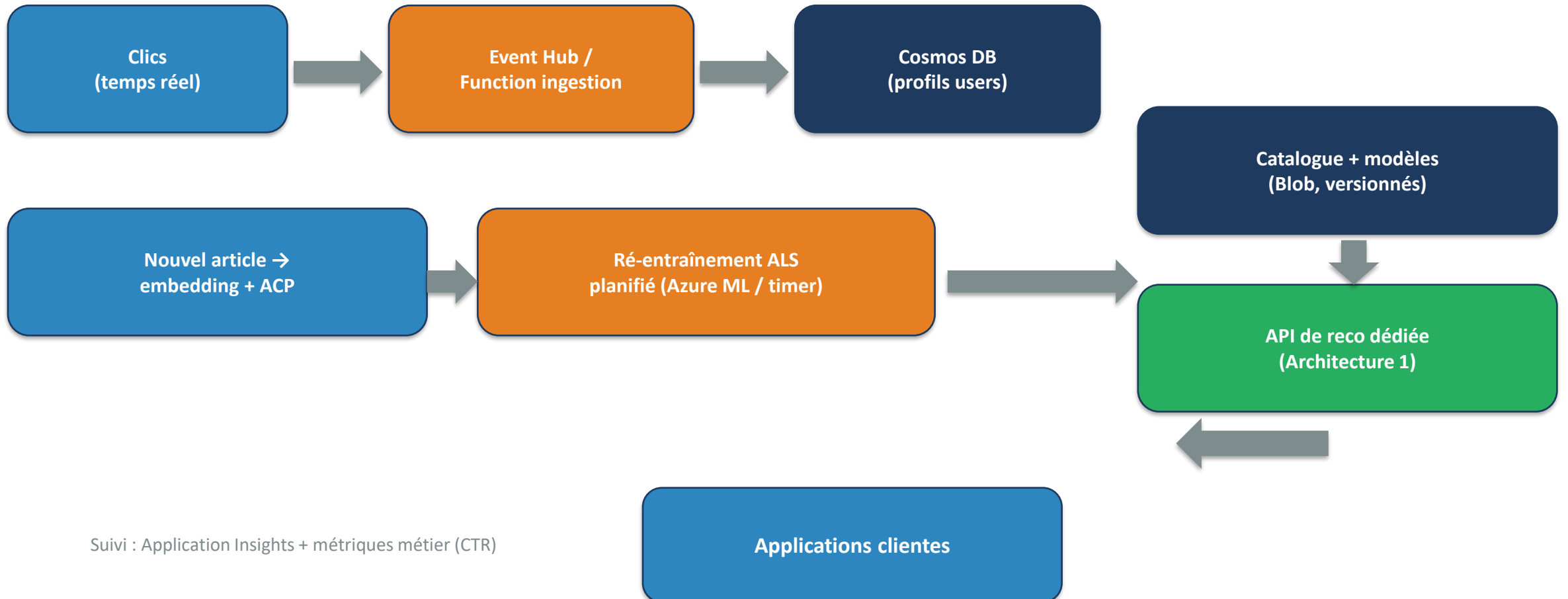
[Capture d'écran
du Space HF
à insérer]

Industrialisation & déploiement bout-en-bout

- Code versionné : Git local + push GitHub.
- Séparation claire : préparation hors-ligne / service d'inférence / application.
- Déploiement reproductible, deux cibles depuis les mêmes artefacts :
 - `prepare_model.py` → artefacts (source commune)
 - Azure : `func azure functionapp publish` (+ artefacts vers Blob)
 - Hugging Face : push du Space + `upload_artifacts.py` vers le HF Hub
- Support de présentation lui-même généré par script (reproductible).

Architecture cible — nouveaux utilisateurs & articles

Intégration en continu, sans tout recalculer



Cible — comment sont gérés les nouveaux venus

- Nouvel article :
 - calcul de son embedding (+ ACP) → disponible immédiatement via le content-based, sans ré-entraînement.
- Nouvel utilisateur :
 - popularité au tout début, puis content-based dès le 1^{er} clic ; profil enrichi en continu via l'ingestion.
- Modèle collaboratif :
 - ré-entraînement planifié (batch) ; entre deux, repli propre sur content-based / popularité.
- Montée en charge :
 - passage à l'API dédiée (Archi 1) : mise à l'échelle, cache, A/B testing, versionnage des modèles.

Limites connues & prochaines étapes

- Évaluation à approfondir : MAP@k, couverture, diversité (au-delà du HitRate@5).
- Prise en compte de la fraîcheur / récence des articles.
- Sécurité : clé de fonction (MVP) → authentification robuste à terme.
- Passage progressif à l'ingestion événementielle et au ré-entraînement automatisé.
- Collecte de nos propres données d'usage pour affiner les modèles.

MVP fonctionnel, serverless et industrialisable

Prêt à démontrer les 5 recommandations et à évoluer vers l'architecture cible.